Spark-X2.5 端侧模型创新挑战赛 初赛作品

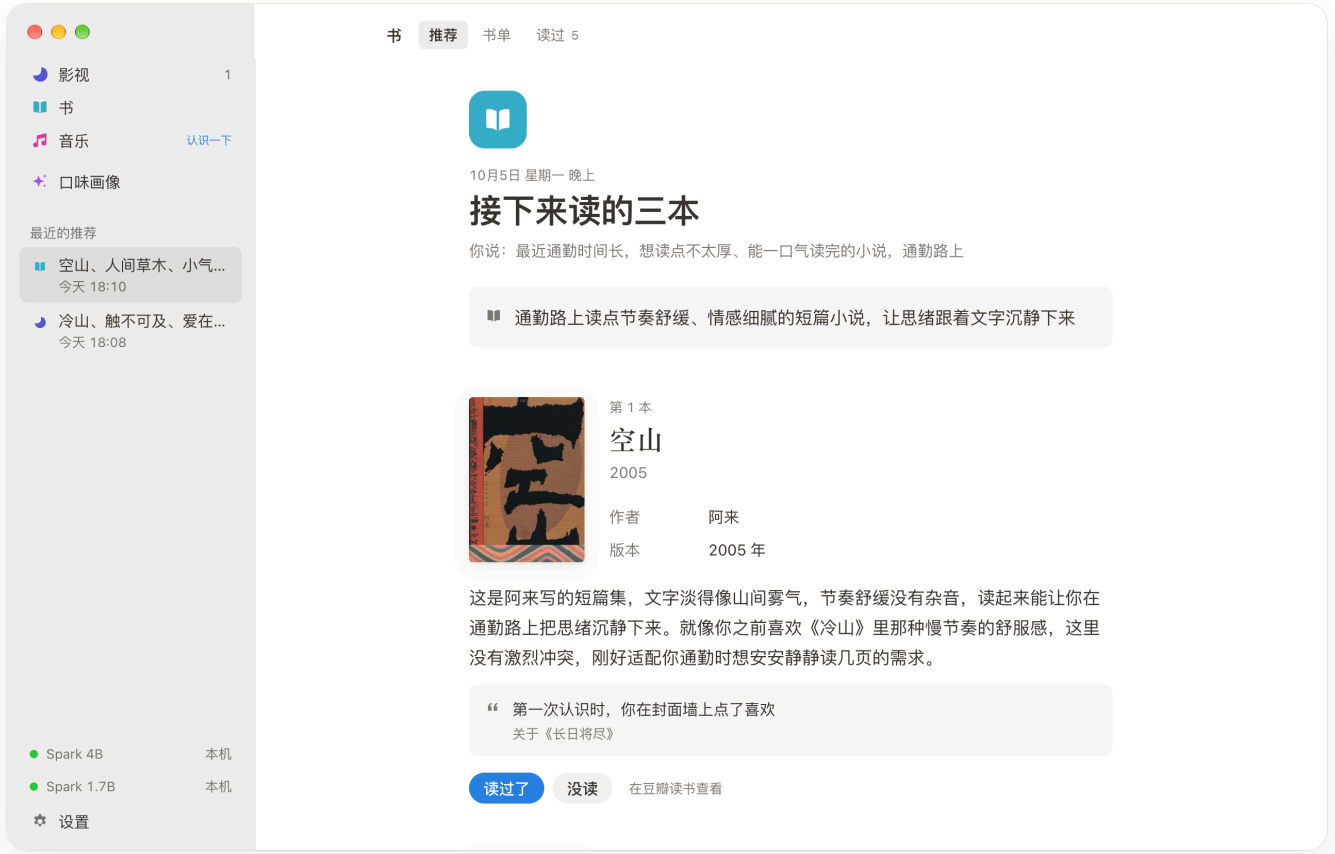
片刻

Fewer films, better nights.

Mac 上的书影音推荐应用。每次只推荐三部，说明理由。两个 Spark-X2.5 模型在本机运行，你的话、评分和口味画像不经过第三方。

Spark-X2.5-4B 与 1.7B　macOS 本机推理

影视 书 音乐　队伍 天天开心



真实界面：推荐《空山》的理由，用上了你看完《冷山》写下的感受

吕浩博

天津师范大学管理学院

2026 年 10 月

只推荐三部，每一部都说清为什么

片刻是 Mac 上的书影音推荐应用，两个 Spark-X2.5 模型都在本机运行

3 部

每次只推荐三部，不做信息流

理由里引用你自己说过的话，原话从本机数据库逐字读出

79%

4B 说出的片名能在片库查到

1.7B 只有 19%。挑选交给 4B，每部作品仍要先核实

1.4 秒

12 万 token 的记录命中缓存后读完

首次读入 726 秒。推荐只读画像，整理画像才读全部记录

你的话、评分和画像只存在这台 Mac 上，联网请求里只有作品名、作者、年份或公开编号

说一句今天的状态 › 4B 读画像定方向 › 本地库核实候选 › 4B 挑三部 › 1.7B 整理评语 › 画像更新

01

项目背景

片刻要解决什么问题，为什么这件事适合放在端侧

1 推荐越准，越需要私人上下文

2 你的信息留在本机，联网只查作品

推荐越准，越需要私人上下文

现有推荐方式的三个问题，以及片刻的回答

1

按停留时长优化

信息流抓住的是此刻还想继续刷的冲动，不是看完觉得值。Kleinberg、Mullainathan 和 Raghavan 在 2022 年专门讨论过这种偏好不一致。

2

口味说不清，只能靠点击猜

协同过滤要在同一个书架上积累大量行为才能学到偏好，读不懂我喜欢它什么都没说出口这样的话，也没法把电影里的口味带到书上。

3

准和私互相矛盾

看过什么、为什么喜欢、当时累不累、和谁一起，这些正是推荐需要的上下文，也都是很私人的信息，交给云端并不合适。

片刻的回答

每次只推荐三部，附上理由。看完以后再问你值不值，下一次推荐会读到你的回答。

片刻的回答

大语言模型直接读你的评语和对话，口味画像三个书架共用，单列一节书影音之间。

片刻的回答

模型和记忆都放在这台 Mac 上。联网只查公开的作品信息和封面，请求里没有关于你的内容。

你的信息留在本机，联网只查作品

全部出站请求从同一个模块发出，模块自己拼 URL，调用方只能传作品名这类字符串

留在这台 Mac 上

你的原话

冷启动对话、评语、没看的原因，逐字保存

星级、日期、状态

每部作品一条记录，可在记录页编辑

口味画像与近况

四层记忆，存在本机 SQLite

Spark-X2.5-4B 与 1.7B

llama.cpp 在本机运行，只监听 127.0.0.1

lookup/net.py

唯一的出站模块

缓存　按站点限速　连不上就暂停

源码原文 `backend/pianke/lookup/net.py`

```
"""The only place 片刻 talks to the internet.

Rule: a request may carry a work's title,
creator, year or a public id, and
nothing else. No user words, ratings, profile
text, names or identifiers ever
leave the machine, and there is no account.
Callers pass plain strings; this
module builds the URLs itself so nothing else
can slip in.
```

发往公开来源

请求里只有

作品名　作者　年份　IMDb 等公开编号

从不包含

原话　星级　画像　姓名　设备标识

封面　豆瓣、维基百科、Open Library、Cover Art Archive、iTunes

核实　书库或曲库里没有时，按书名或专辑名查豆瓣读书和 iTunes

没有账号。设置里可以关掉联网封面，界面改用文字封面。

02

产品功能

三个书架共用一套流程：冷启动、推荐、反馈、回访、记录与画像

1 三个书架
2 冷启动
3 一次推荐
4 反馈与回访
5 记录与画像

三个书架，一份口味

影视、书、音乐各有自己的记录和片单，口味画像三个书架共用



首页：说一句今天的状态，选几个标签，片刻只挑三部

影视

电影和剧集。本地片库 32,914 部，来自 IMDb 非商业数据集

书

小说、散文、非虚构。本地书库 3,769 本，来自 Wikidata，库里没有时按书名联网核实

音乐

以专辑为单位。本地曲库 5,575 张，来自 Wikidata，含华语和日韩专辑

口味画像

分五节：你偏爱的、你会避开的、书影音之间、你的习惯、最近的变化。每条结论都标出依据

冷启动：先认封面，再聊几句

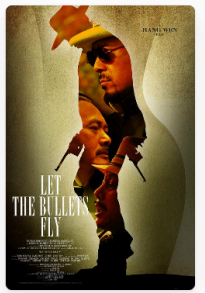
认出看过的作品比凭空回忆省力，点过的每一部都已经是核实过的作品

1 选书架 可以多选，没选的以后点开时再认识

- **影视** 电影和剧集。今晚看什么，只挑三部。 ✓
- **书** 小说、散文、非虚构。下一本读什么。 ✓
- **音乐** 整张专辑。工作、通勤、睡前听什么。

2 封面墙 点一下是喜欢，点两下是不喜欢

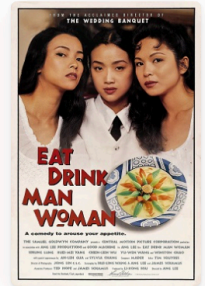

寄生虫
2019



让子弹飞
2010



流浪地球
2019



饮食男女
1994



功夫
2004

3 聊几句 4B 从你点过的作品问起

你点了《寄生虫》、《流浪地球》和《功夫》。挑一个说说，它哪里打动你？

> 长日将尽吧。管家一辈子把感情压在职责下面，到最后才露出一点点，那种克制我特别喜欢，和花样年华有点像。

我听到你偏爱《长日将尽》里那种克制的情感，觉得它像《花样年华》一样，能让人感受到深藏心底的细腻。

你对《落水狗》点了不喜欢。是哪里不对胃口？

4 第一版画像 4B 提到的作品逐一核实，结论标出依据

你偏爱克制的情感，排斥吵闹血腥与拖沓内容，偏好独处观影与通勤阅读。

你偏爱的

- 读书时偏爱《长日将尽》里压在职责下的克制情感，觉得深藏心底的细腻动人。
  长日将尽

一次推荐：读画像，定方向，核实，挑三部

四步都在本机完成，界面如实显示每一步在做什么



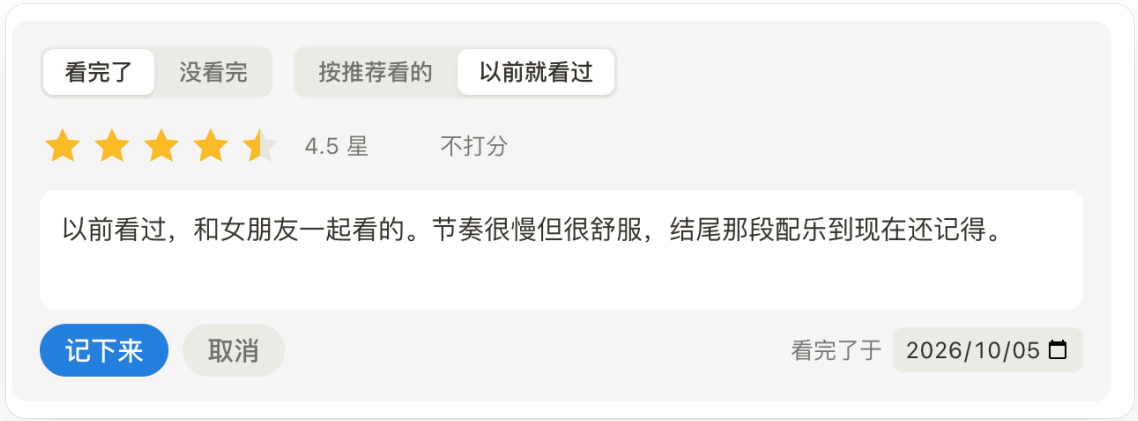
定方向	4B	把画像和今晚的要求变成筛选条件，JSON 语法只允许这个书架真实存在的类型、语种和地区
核实		模型想到的每部作品先查本地库，再按名字联网查，查不到的直接丢掉
挑三部	4B	id 只能取自候选清单；理由里的因为你说过，只能指向你自己的记录



每一部推荐都有下文

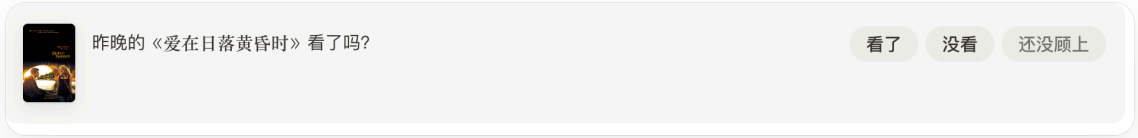
看过了就打星写评语，没看就选一个原因，过几天片刻会回来问一次

看过了 1.7B

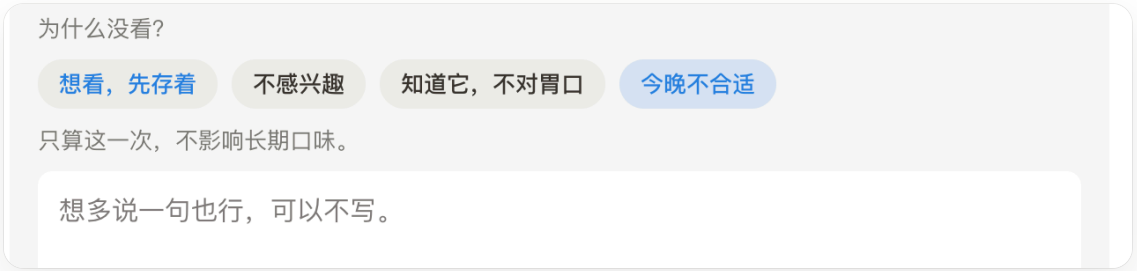


0 到 5 星，可以拖动，精确到半星，星级只由你给。1.7B 从评语里整理喜欢和不喜欢的方面，原话逐字保存。

第二天回访 加进片单的，或那晚一部都没碰的，下次打开时问一次



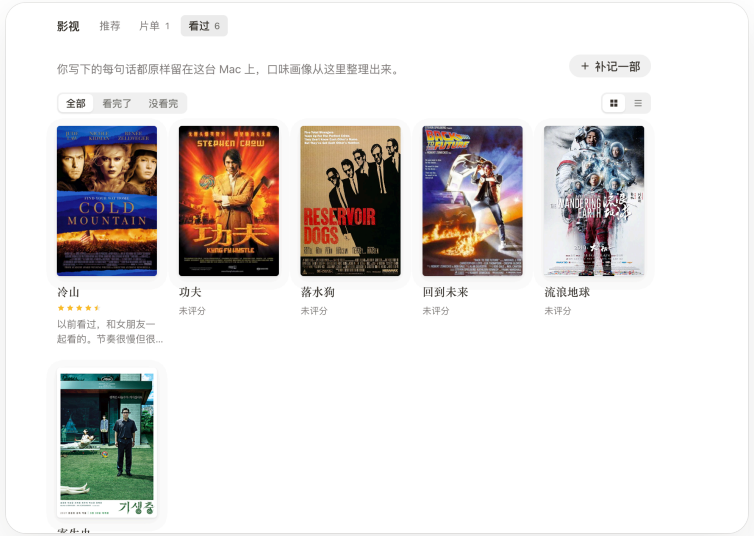
没看 选一个原因，每个原因写到不同的地方



想看，先存着	进片单、书单或歌单，之后会回访
不感兴趣	带原因的负面记录，整理画像时会读
知道它，不对胃口	
今晚不合适	只进最近两周的近况，长期画像看不到

记录可以编辑，画像有据可查

每个书架一份日志，点开是像 Notion 一样可编辑的记录页；画像每条结论都标出依据



记录 画廊或表格两种视图，按看完、没看完筛选



记录页 状态、星级、日期、来由、评语都能改；改了评语，1.7B 会重读一遍



口味画像 4B 读完全部记录后重写，引用的作品逐条校验，你改过的条目永远保留

03

技术实现

采用的技术、产品架构、两个模型的分工，以及在真机上的实测

1 技术架构
2 两个模型的分工
3 分层记忆与长上下文
4 对比实验
5 端侧实测与工程
6 创意与前瞻

技术架构：全部在一台 Mac 上

Electron 界面、FastAPI 后端、两个 llama.cpp 模型服务和四层 SQLite 记忆，出站请求只有一个口

界面

Electron 桌面端

React、Vite、Tailwind；颜色与图标统一来自一份设计 token

- 冷启动与封面墙
- 首页与推荐结果
- 看过了与没看
- 记录页与画像

SSE
127.0.0.1

后端 FastAPI，只监听 127.0.0.1

流程

冷启动访谈、推荐四步、反馈、回访、整理画像，三个书架共用

4B

llama-server，Q4_K_M，上下文 128K，Metal

1.7B

llama-server，Q4_K_M，上下文 16K，Metal

记忆 user.sqlite

L0 事件、L1 记录、L2 画像、L3 近况

片库

IMDb，32,914 部

书库与曲库

Wikidata，3,769 本与 5,575 张

只有作品名

出站

lookup/net.py

唯一的出站模块；缓存在 cache.sqlite，只存公开数据，可随时删除

公开来源

豆瓣、维基百科、Wikidata、Open Library、Cover Art Archive、iTunes

联网失败时

请求失败和确实没有分开记；连不上的站点暂停 5 分钟，界面改用文字封面

两个模型的分工

需要理解口味的交给 4B，高频、低风险、要快的交给 1.7B；所有输出都走 JSON 语法约束

模型	任务
4B	冷启动对话的回应，抽出对话里提到的作品
4B	定方向：画像和今晚的要求 → 片库筛选条件
4B	从候选清单里挑三部，写理由，标出依据的记录
4B	整理画像：一次读完全部记录，重写五节画像
1.7B	把评语整理成结构化记录，平均 0.92 秒

为什么选 Spark-X2.5

4B 在 18GB 内存的 Mac 上能读完 12 万 token 的记录；同系列两个尺寸用同一套提示词和输出格式，可以按任务的难度分配，也方便做替换对比。

挑三部的输出格式：id 只能取自候选清单 `flows/recommend.py`

```
"picks": {
    "type": "array", "minItems": 3, "maxItems": 3,
    "items": {
        "type": "object",
        "properties": {
            "id": {"type": "string", "enum": [t["id"] for t in pool]},
            "name": {"type": "string"},
            "reason": {"type": "string"},
            "because": (
                {"type": "array", "maxItems": 2, "items": {"type":
"string", "enum": rel_ids}}
                if rel_ids else {"type": "array", "maxItems": 0, "items":
{"type": "string"}}
```

评语里没说的心情和同伴，一律丢掉 `flows/feedback.py`

```
# context the user never said is dropped: it must share two characters in
# a row with their words (the 1.7B model likes to infer a mood)
for k in ("mood", "with"):
    v = data[k]
    if v and not any(v[i : i + 2] in text for i in range(max(1, len(v) -
1))):
        data[k] = ""
```

推荐只读画像，长上下文留给整理

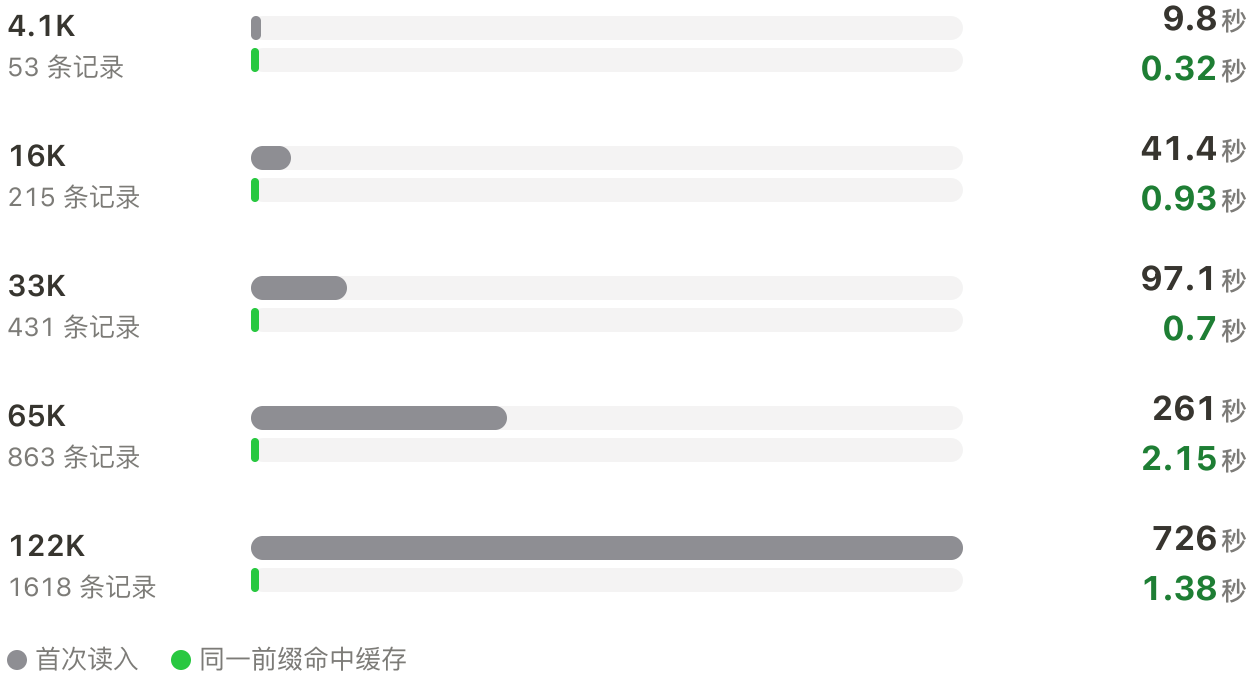
记录从 50 条涨到 5000 条，推荐的输入长度不变；整理画像时 4B 把全部记录完整读一遍

层	内容	谁读
L0 事件	说过、做过的一切，只追加	审计与导出
L1 记录	每部作品一条：星级、方面、评语原话	整理画像时全部读
L2 画像	五节口味画像，每条附依据，有版本	每次推荐
L3 近况	最近两周，含今晚不合适跳过的	每次推荐

推荐时另带几条和候选相关的记录，同书架为主，再带几条别的书架里感受最强的。

4B 读入全部记录的耗时

M3 Pro 18GB，Q4_K_M，llama.cpp Metal



首次读完 12.2 万 token 要 12 分钟，所以画像整理放在不赶时间的时候做；同一份记录再读一次只要 1.4 秒。

为什么挑选交给 4B，整理评语交给 1.7B

同一套提示词，分别换上两个模型实测；数据见 data/eval_ablation.json，脚本 eval_ablation.py 可复跑

直接说出片名，能在片库里查到的比例



1.7B 说出的多是不存在的片，例如 春日花开 2022、海边的图书馆 2023。4B 也有查不到的，所以每部作品都要先核实，查不到就丢掉。

id 约束

挑三部时不加约束和加枚举约束各跑 24 次，越界都是 0。约束保留下来当保险，换模型也不会推荐清单以外的作品。

把评语整理成结构化记录，12 条

指标	1.7B	4B
判断看完与否	83%	92%
星级误差在 1 星以内	83%	100%
原话逐字保留	100%	100%
平均用时	0.92 秒	1.84 秒

评语整理每条记录都要做，用 1.7B 快一倍。看完与否以你按的按钮为准，星级以你自己打的为准，模型的判断只在你没给时补位，所以 1.7B 的误差不会覆盖你的选择。

端侧实测与工程质量

数字都来自这台测试机和仓库里的脚本，可以复跑

在这台 Mac 上

测试机	Apple M3 Pro，18GB 内存
模型文件	4B 2.6 GB，1.7B 1.1 GB，均为 Q4_K_M 量化
推理	llama.cpp，Metal 加速，flash attention
一次推荐	4B 用时 71.1 秒，见下方界面原文
整理一条评语	1.7B 平均 0.92 秒
生成速度	4B 在 4K 上下文时 27.8 token/秒

由这台 Mac 上的 Spark-X2.5-4B 挑选，用时 71.1 秒。你的记录没有离开这台 Mac。

换个说法　再挑三部

推荐结果底部的说明，用时由后端如实记录

工程

16 个后端测试全部通过

覆盖作品核实与拒绝编造的片名、没看原因的去向、半星评分、回访只问一次、你改过的画像条目保留

可复现

build_catalog.py 和 build_library.py 从公开数据建库；bench_speed.py 和 eval_ablation.py 复跑本页数字；驱动脚本在真机上自动操作界面，PPT 里的界面图都由它拍摄

出错时

模型输出走 JSON 语法约束，引用的记录 id 逐条校验；请求失败和确实没有分开记；模型没启动时界面如实显示状态

一键运行

run.sh 构建界面并打开应用，后端和两个模型由应用自动拉起

创意与前瞻：口味可以跨书架迁移

在电影里喜欢克制叙述的人，读小说时多半也是；大语言模型读得懂这种说法

书影音之间

推荐书时会带上几条电影和音乐里感受最强的记录。下面是真机上的一次书推荐，理由用上了你刚给《冷山》写的评语。

这是阿来写的短篇集，文字淡得像山间雾气，节奏舒缓没有杂音，读起来能让你在通勤路上把思绪沉静下来。就像你之前喜欢《冷山》里那种慢节奏的舒服感，这里没有激烈冲突，刚好适配你通勤时想安安静静读几页的需求。

少而准的推荐

每次三部，理由引用你自己的话；没看的原因分四种去向，今晚不合适不会被当成不喜欢。

端侧让私人上下文可以放心用

累了、和谁一起、为什么弃了，这些话说给本机的模型听，不用担心被拿去做别的事。

下一步

手机端	同一份记忆在手机上运行，需要移动端推理 SDK
拍照建档	拍书脊、海报或唱片封套加入记录，需要小尺寸视觉模型
画像缓存落盘	打开 llama.cpp 的 slot 保存，重启后整理画像不必从头读
更快的长上下文	12 万 token 首次读入 12 分钟，希望有官方 Metal 优化或 MLX 版本
更多书架	播客、游戏、展览，沿用同一套推荐管线

04

团队与声明

团队介绍、合作资源与项目原创性声明

1 团队介绍
2 项目原创性声明

团队介绍

天天开心，学生团队，未注册公司，未融资

吕浩博

主负责人

现在	天津师范大学管理学院，博士研究生，信息资源管理方向
此前	大连外国语大学，图书情报硕士，2026 年毕业
研究	大模型智能体、多智能体检索增强生成、大模型引用幻觉的检测与抑制

片刻要求模型提到的每部作品都先核实，这个设计来自对大模型幻觉的研究积累。

团队架构

模型与推荐管线

提示词与输出格式、推荐四步、画像整理

桌面端与交互

Electron 界面、冷启动、反馈与记录页

数据与评测

片库、书库、曲库，测速与对比实验

合作伙伴资源

暂无商业合作伙伴。作品数据来自公开数据集 IMDb 非商业数据集与 Wikidata，推理基于开源的 llama.cpp。

项目原创性声明

天天开心团队就参赛作品片刻作如下声明

本团队承诺，参赛作品片刻的方案与成果均为团队原创，内容真实。作品中使用的第三方数据与开源软件均按其许可使用，并在本页列明。如发生知识产权纠纷，由参赛者自行承担责任。

天天开心 吕浩博　　2026 年 10 月

致谢与第三方资源

模型	Spark-X2.5-4B、Spark-X2.5-1.7B
推理	llama.cpp
片库	IMDb 非商业数据集
书库与曲库	Wikidata，CC0
封面与核实	豆瓣、维基百科、Open Library、Cover Art Archive、iTunes Search API
界面	Electron、React、Vite、Tailwind CSS